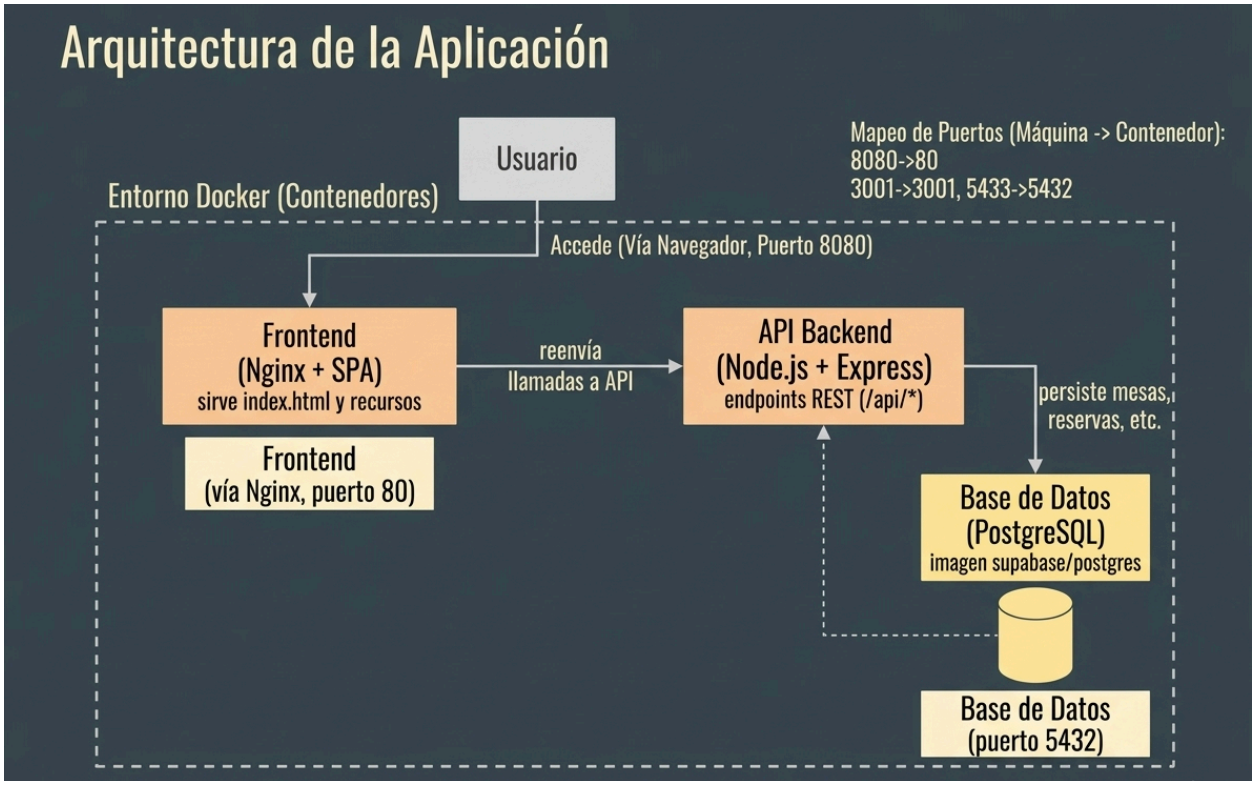
Entregable del Taller de Docker Compose

2.Diseño del Sistema

El sistema distribuido propuesto se compone de tres servicios principales: un **Frontend**, un **API Backend** y una **Base de Datos**.



4. Análisis de la Implementación

4.1. Rol de Cada Servicio

- **Frontend (Nginx):** Actúa como la puerta de entrada para los usuarios, sirviendo la interfaz de usuario estática y dirigiendo las solicitudes de la API al backend.
- **API Backend (Node.js/Express):** Contiene la lógica de negocio principal, procesa las solicitudes de la API, realiza validaciones y gestiona la interacción con la base de datos.
- **Base de Datos (PostgreSQL):** Es el componente de persistencia, encargado de almacenar de forma segura todos los datos de la aplicación, como mesas, reservas y días disponibles.

4.2. Ventajas de Dividir el Sistema

Dividir el sistema en servicios independientes ofrece varias ventajas significativas:

- **Modularidad:** Cada servicio se enfoca en una tarea específica, lo que facilita su desarrollo, mantenimiento y comprensión.
- **Escalabilidad independiente:** Los servicios pueden escalarse de forma individual según la demanda. Por ejemplo, si el backend recibe muchas solicitudes, se pueden añadir más instancias de este servicio sin afectar al frontend o la database.
- **Resiliencia:** Si un servicio falla, los otros pueden seguir funcionando (o degradarse de forma controlada), evitando que una falla en un componente derribe todo el sistema.
- **Tecnologías Variadas:** Permite utilizar diferentes tecnologías y lenguajes de programación para cada servicio, eligiendo la herramienta más adecuada para cada tarea.
- **Despliegue Continuo:** Facilita la actualización y el despliegue de servicios de forma independiente, reduciendo el riesgo y el tiempo de inactividad.

4.3. Comunicación entre Contenedores

Los contenedores se comunican principalmente a través de la red interna de Docker Compose. Cuando se define un docker-compose.yml, Docker crea una red virtual por defecto a la que se conectan todos los servicios. Esto permite que los servicios se refieran entre sí por sus nombres de servicio (ej. backend se conecta a database usando el nombre database como host). El frontend (Nginx) también utiliza esta red interna para reenviar solicitudes al backend.